بسم الله الرحمن الرحیم

دانشگاه بین‌المللی امام خمینی (ره)

دانشکدهٔ فنی و مهندسی — گروه مهندسی برق

گزارش کار آزمایشگاه — شمارهٔ ۳

کنترل دو LED به‌صورت مجزا با آردوینو و زمان‌بندی مستقل خروجی‌های دیجیتال

پیاده‌سازی حلقهٔ for، متغیرهای سراسری، کار با دکمه، سنسور دما و زمان‌بندی با millis() در محیط Tinkercad Circuits

نام دانشجو	**مهدیه غلامی**
شمارهٔ دانشجویی	**۴۰۱۶۱۲۳۰۵۸**
نام استاد	**جناب آقای مهندس سلیمانی الموتی**
گروه	**۶۰**
درس	**آزمایشگاه سیستم‌های دیجیتال**
تاریخ	**۱۵ آبان ۱۴۰۵**

۱۵ آبان ۱۴۰۵

چکیده

در این گزارش، کنترل دو LED به‌صورت مجزا و مستقل با استفاده از میکروکنترلر آردوینو بررسی شده است. ابتدا فرمان‌های پایه مانند `digitalWrite`، `pinMode` و `delay` مرور و سپس مفهوم حلقهٔ `for` برای تکرار دستورات، متغیرهای سراسری برای کنترل پارامترها و ریل Ground روی بردبورد معرفی می‌شود. در ادامه، دوازده تکلیف عملی شامل کنترل مستقل دو LED، شمارش با حلقه، تعامل با دکمه، خواندن سنسور دما و زمان‌بندی هم‌زمان با `millis()` طراحی و در محیط شبیه‌سازی Tinkercad Circuits پیاده‌سازی گردیده‌اند. نتایج نشان می‌دهد که تمامی مدارها مطابق رفتار مورد انتظار عمل می‌کنند و هر خروجی به‌خوبی به‌صورت مستقل از دیگری قابل کنترل است.

کلیدواژه‌ها: آردوینو، کنترل مستقل LED، حلقهٔ for، متغیر سراسری، ریل Ground، دکمهٔ فشاری، سنسور دما، millis()

فهرست مطالب

۱ مقدمه
۲ هدف آزمایش
۳ مبانی: digitalWrite، delay و حلقهٔ for
۴ متغیرهای سراسری و ریل Ground
۵ شرح قسمت عملی آزمایش
۶ حل المسائل (تکالیف ۱ تا ۲)
۷ نتایج شبیه‌سازی
۸ تحلیل نتایج
۹ نتیجه‌گیری
۱۰ پیشنهادات
۱۱ منابع

۱. مقدمه

در جلسات پیش آموختیم چگونه یک یا چند LED را به‌صورت هم‌زمان روشن و خاموش کنیم. در این جلسه، هدف کنترل روشن و خاموش‌شدن دو LED به‌صورت مجزا و مستقل از یکدیگر است. برای این کار از دو پین جداگانهٔ آردوینو استفاده و با بهره‌گیری از فرمان‌های `digitalWrite()` و `delay()` رفتار هر LED مستقل تنظیم می‌شود.

برای ساخت مدار، دو LED و دو مقاومت به دو پین دیجیتال متصل می‌گردند. یک ریل Ground از پین GND آردوینو به یکی از ردیف‌های کناری بردبورد وصل می‌شود؛ سپس هر قطعه‌ای که به آن ردیف متصل شود، مانند اتصال به زمین عمل می‌کند. این ریل، اتصالات پیچیده را کاهش داده و وصل‌کردن تعداد زیادی قطعه را ساده می‌سازد.

۲. هدف آزمایش

۱ کنترل مستقل دو LED روی دو پین متفاوت آردوینو.

۲ به‌کارگیری حلقهٔ `for` برای تکرار دقیق تعداد دفعات چشمک.

۳ استفاده از متغیرهای سراسری برای پارامترهای قابل‌تنظیم.

۴ تعامل با دکمه، خواندن سنسور دما و زمان‌بندی با `millis()` .

💡 نکته: تعریف متغیرها بیرون از توابع (سراسری) باعث می‌شود آن‌ها در کل برنامه قابل دسترس باشند و تنها با تغییر یک مقدار، رفتار کل مدار به‌روز شود.

۳. مبانی: digitalWrite، delay و حلقهٔ for

`digitalWrite(pin, val)`

وضعیت پین خروجی را روی `HIGH` (روشن) یا `LOW` (خاموش) می‌برد و پایهٔ کنترل مستقل هر LED است.

`delay(ms)`

به مدت مشخصی (میلی‌ثانیه) توقف ایجاد می‌کند و مدت روشن یا خاموش بودن هر LED را تعیین می‌نماید.

`for (init; test; step)`

برای تکرار یک بلوک دستورات به تعداد معین به کار می‌رود؛ مثلاً چشمک‌زدن ده‌بارهٔ یک LED.

`millis()`

زمان سپری‌شده از شروع برنامه را برمی‌گرداند و امکان زمان‌بندی هم‌زمان چند LED را بدون توقف برنامه فراهم می‌کند.

متغیرهای سراسری و ریل Ground

برای دسترسی به شمارهٔ پین‌ها و تعداد دفعات در کل برنامه، آن‌ها را به‌صورت متغیر سراسری (Global) و بیرون از توابع تعریف می‌کنیم. در طرف سخت‌افزار نیز یک ریل Ground از پین GND به ردیف کناری بردبورد متصل می‌شود تا پایانهٔ مشترک همهٔ قطعات را به‌سادگی فراهم آورد.

```
int redLED = 9;         // پین متغیر سراسری: LED قرمز
int yellowLED = 10;     // پین متغیر سراسری: LED زرد
int blinkCount = 5;     // تعداد دفعات چشمک (قابل تغییر)
```

۴. معرفی برد آردوینو و بردبورد

در برد آردوینو Uno، پین‌های ۰ تا ۱۳ برای ورودی و خروجی دیجیتال در دسترس‌اند و در این آزمایش عمدتاً از پین‌های ۹ و ۱۰ برای دو LED استفاده شده است. پین‌های آنالوگ A0 تا A5 برای خواندن سنسور دما به کار می‌روند و پین GND نیز ریل زمین بردبورد را تأمین می‌کند.

پین‌های دیجیتال ۹ و ۱۰

خروجی کنترل LED قرمز و زرد؛ هرکدام با یک مقاومت محدودکنندهٔ جریان به زمین متصل می‌شوند.

پین ورودی ۲ (دکمه)

با پیکربندی `INPUT_PULLUP` حالت فشرده دکمه برابر LOW است و مقاومت خارجی لازم نیست.

پین آنالوگ A0 (سنسور دما)

خروجی سنسور LM35/TMP36 را با `analogRead` می‌خواند.

ریل Ground

سیم از پین GND به ردیف کناری بردبورد؛ همهٔ اتصالات منفی از این ریل استفاده می‌کنند.

۵. شرح قسمت عملی آزمایش

به دلیل در دسترس نبودن فیزیکی برد، تمامی مدارها در محیط شبیه‌سازی آنلاین **Tinkercad Circuits** طراحی و اجرا شده‌اند. روش کار در هر تکلیف به شرح زیر است:

1. قرار دادن برد آردوینو Uno و بردبورد و اتصال ریل Ground از پین GND.
2. افزودن دو LED با مقاومت (و در صورت نیاز دکمه یا سنسور دما) و سیم‌کشی مطابق صورت مسئله.
3. تعریف متغیرهای سراسری پین‌ها و تنظیم آن‌ها به‌عنوان خروجی در تابع `setup` .
4. اجرای شبیه‌سازی با دکمهٔ Start و مشاهدهٔ رفتار مستقل LEDها و Serial Monitor.

⚡ نکتهٔ ایمنی: پایهٔ بلند LED (آند) به پین خروجی و پایهٔ کوتاه (کاتد) از طریق یک مقاومت حدود ۲۲۰ اهم به ریل Ground متصل می‌شود تا جریان از حد مجاز فراتر نرود.

مسئلهٔ ۱ کنترل دو LED به‌صورت مجزا

صورت مسئله

مداری طراحی کنید که دو LED به پین‌های مختلف آردوینو متصل شده باشند؛ این دو LED باید به‌طور مستقل از یکدیگر روشن و خاموش شوند.

نکات کلیدی

- هر LED به یک پین دیجیتال مستقل (پین ۹ و پین ۱۰) متصل شده و در `setup` به‌عنوان خروجی پیکربندی می‌شود.
- دستور `digitalWrite` سطح منطقی هر پین را جداگانه روی HIGH یا LOW می‌برد.
- با `delay(1000)` هر LED یک ثانیه روشن و یک ثانیه خاموش می‌ماند تا رفتار مستقل آن‌ها دیده شود.

کد آردوینو

```
int redLED = 9;
int yellowLED = 10;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
}

void loop() {
  digitalWrite(redLED, HIGH);
  delay(1000);

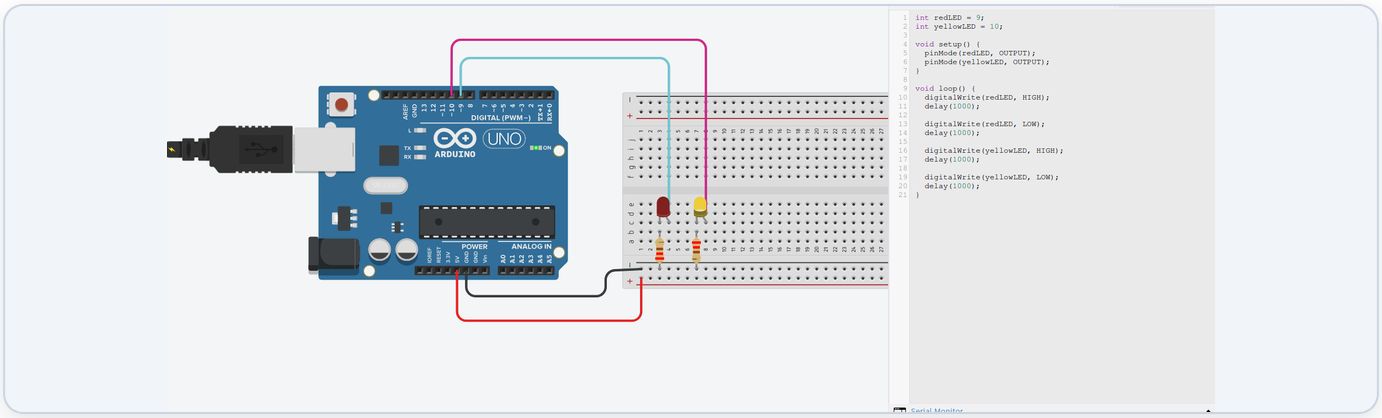
  digitalWrite(redLED, LOW);
  delay(1000);

  digitalWrite(yellowLED, HIGH);
  delay(1000);

  digitalWrite(yellowLED, LOW);
  delay(1000);
}
```

توضیح عملکرد

در این مدار، دو LED قرمز و زرد روی پین‌های ۹ و ۱۰ نشان داده شده‌اند و یک ریل Ground روی بردبورد، پایانهٔ منفی هر دو LED را به زمین آردوینو متصل می‌کند. چون فرمان‌های روشن و خاموش‌کردن برای هر LED به‌صورت پیاپی و مستقل نوشته شده‌اند، در هر لحظه فقط یک پین تغییر وضعیت می‌دهد و LED دیگر بدون تأثیر باقی می‌ماند. این ساده‌ترین نمونهٔ کنترل مستقل خروجی‌های دیجیتال است.



شکل ۱: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۲ چشمک زدن LED قرمز ۱۰ بار و سپس LED زرد یک بار

صورت مسئله

LED قرمز باید ۱۰ بار چشمک بزند و سپس LED زرد یک بار چشمک بزند. برای تکرار دستورات از حلقهٔ `for` استفاده کنید.

کد آردوینو

```
int redLED = 9;
int yellowLED = 10;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
}

void loop() {
  for (int i = 0; i < 10; i++) {
    digitalWrite(redLED, HIGH);
    delay(500);
    digitalWrite(redLED, LOW);
    delay(500);
  }

  digitalWrite(yellowLED, HIGH);
  delay(500);

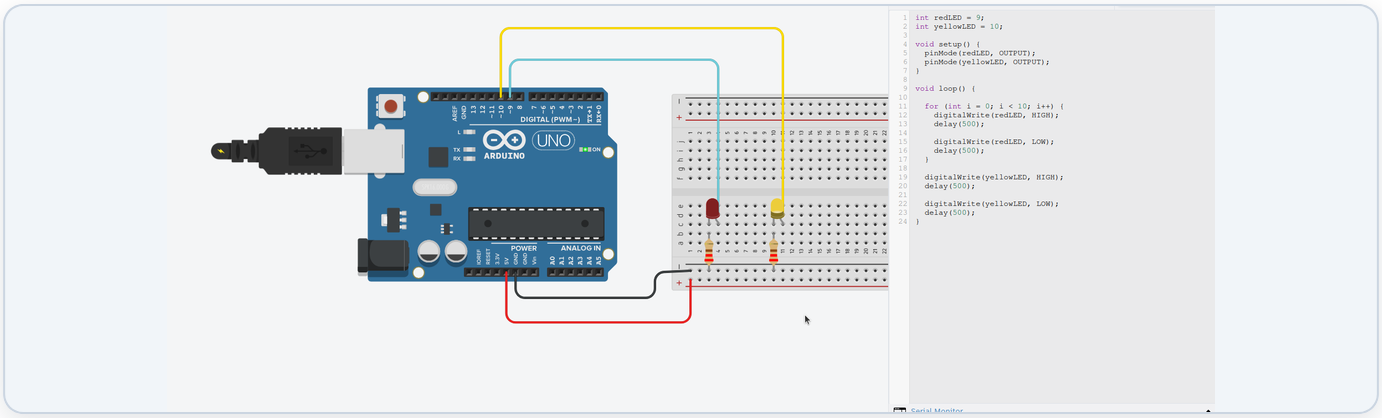
  digitalWrite(yellowLED, LOW);
  delay(500);
}
```

نکات کلیدی

- حلقهٔ `for (int i = 0; i < 10; i++)` دقیقاً ده بار اجرا می‌شود و LED قرمز را ده بار چشمک می‌کند.
- پس از پایان حلقه، دستورات مربوط به LED زرد اجرا می‌شوند تا تنها یک بار روشن و خاموش شود.
- تأخیر ۵۰۰ میلی‌ثانیه‌ای، ریتم چشمک‌زدن را تندتر و قابل‌تشخیص می‌کند.

توضیح عملکرد

این تمرین کاربرد حلقهٔ for برای تکرار یک بلوک دستورات را نشان می‌دهد. تا زمانی که شمارنده i کوچک‌تر از ۱۰ است، بدنهٔ حلقه تکرار شده و LED قرمز روشن و خاموش می‌شود. به‌محض رسیدن i به ۱۰، حلقه پایان می‌یابد و اجرای برنامه به دستورات بعدی می‌رود؛ بنابراین LED زرد فقط یک بار چشمک می‌زند. چون همهٔ این کارها درون loop است، این الگو بی‌پایان تکرار می‌شود.



شکل ۲: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۳ کنترل LED با دکمه (روشن شدن به مدت ۲ ثانیه)

صورت مسئله

یک دکمه را به آردوینو وصل کنید. وقتی دکمه فشرده می‌شود، یک LED روشن شود؛ این LED باید به مدت ۲ ثانیه روشن بماند و سپس خاموش شود.

کد آردوینو

```arduino
int button = 2;
int led = 9;

void setup() {
  pinMode(button, INPUT_PULLUP);
  pinMode(led, OUTPUT);
}

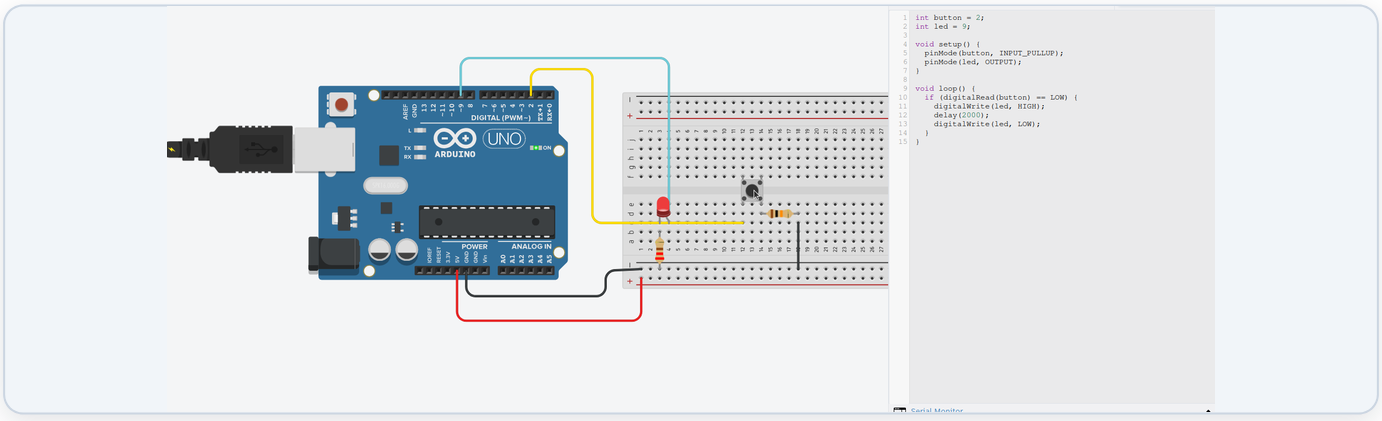
void loop() {
  if (digitalRead(button) == LOW) {
    digitalWrite(led, HIGH);
    delay(2000);
    digitalWrite(led, LOW);
  }
}
```

توضیح عملکرد

در این مدار، دکمه به پین ۲ و LED به پین ۹ متصل شده است. در هر دور از loop وضعیت دکمه با digitalRead خوانده می‌شود؛ اگر کاربر دکمه را فشار داده باشد، ورودی روی LOW رفته و بدنهٔ شرط اجرا می‌گردد. در نتیجه LED روشن شده، دو ثانیه روشن می‌ماند و سپس خاموش می‌شود. این پروژه نخستین گام در جهت تعامل با ورودی‌های دیجیتال و پاسخ‌دهی برنامه به عملکرد کاربر است.

نکات کلیدی

- با `INPUT_PULLUP` پین دکمه در حالت آزاد HIGH و هنگام فشار LOW می‌شود؛ پس شرط با == LOW نوشته شده است.
- به‌محض تشخیص فشردن دکمه، LED روشن و دقیقاً ۲ ثانیه (`delay(2000)`) روشن می‌ماند.
- پس از پایان تأخیر، LED خاموش می‌شود تا چرخهٔ بعدی انتظار فشار دوباره را بکشد.



شکل ۳: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۴ روشن بودن LED قرمز ۵ ثانیه و سپس LED زرد ۳ ثانیه

صورت مسئله

مداری طراحی کنید که در آن یک LED به مدت ۵ ثانیه روشن بماند و سپس خاموش شود؛ پس از آن LED دیگری به مدت ۳ ثانیه روشن شود.

کد آردوینو

```
int redLED = 9;
int yellowLED = 10;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
}

void loop() {
  digitalWrite(redLED, HIGH);
  delay(5000);

  digitalWrite(redLED, LOW);

  digitalWrite(yellowLED, HIGH);
  delay(3000);

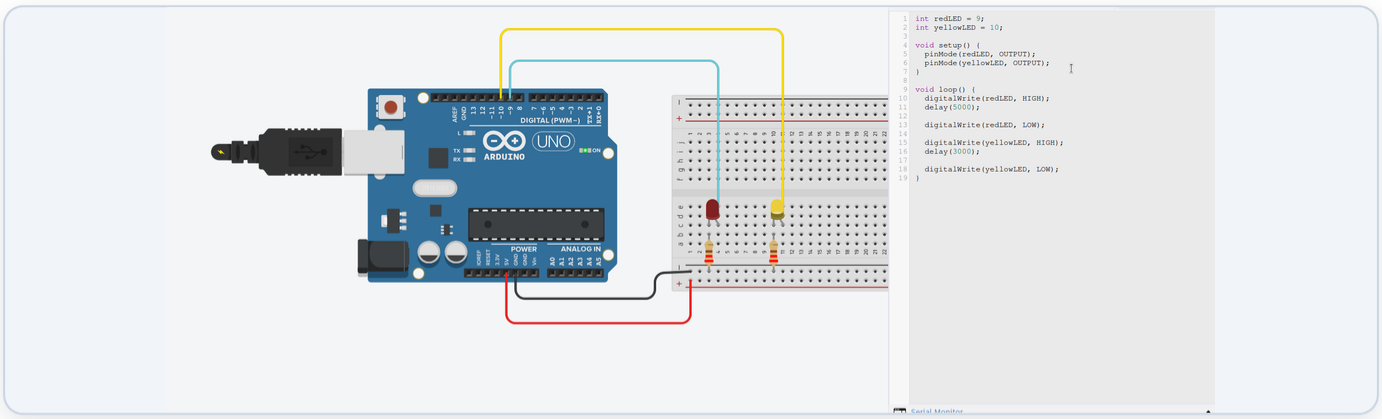
  digitalWrite(yellowLED, LOW);
}
```

نکات کلیدی

- LED قرمز با `delay(5000)` پنج ثانیه روشن می‌ماند و سپس خاموش می‌شود.
- بلافاصله پس از خاموش‌شدن قرمز، LED زرد روشن و سه ثانیه (`delay(3000)`) روشن می‌ماند.
- تغییر زمان‌ها تنها با ویرایش مقادیر delay انجام می‌شود و ساختار کد ثابت می‌ماند.

توضیح عملکرد

این تمرین ترتیب زمانی روشن‌شدن دو خروجی را نشان می‌دهد. در نیمهٔ نخست، پین قرمز روی HIGH رفته و پنج ثانیه صبر می‌کند؛ سپس روی LOW قرار می‌گیرد. در نیمهٔ دوم، پین زرد روشن شده و پس از سه ثانیه خاموش می‌شود. چون این توالی در loop تکرار می‌شود، الگوی «پنج ثانیه قرمز، سپس سه ثانیه زرد» به‌طور پیوسته ادامه می‌یابد.



شکل ۴: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۵ چشمک LED قرمز هر ۵۰۰ میلی‌ثانیه (۵ بار) و سپس LED زرد ۱ ثانیه

صورت مسئله

LED قرمز باید هر ۵۰۰ میلی‌ثانیه روشن و خاموش شود. پس از ۵ بار تکرار، LED زرد به مدت ۱ ثانیه روشن بماند و سپس خاموش شود.

کد آردوینو

```
int redLED = 9;
int yellowLED = 10;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
}

void loop() {
  for (int i = 0; i < 5; i++) {
    digitalWrite(redLED, HIGH);
    delay(500);

    digitalWrite(redLED, LOW);
    delay(500);
  }

  digitalWrite(yellowLED, HIGH);
  delay(1000);

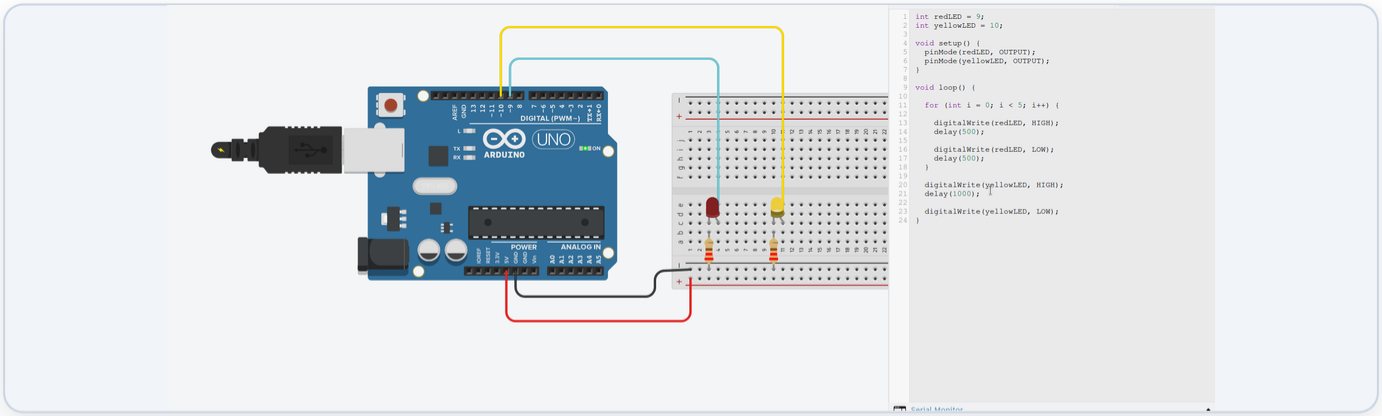
  digitalWrite(yellowLED, LOW);
}
```

نکات کلیدی

- حلقهٔ `for` با حد ۵، دقیقاً پنج دور چشمک برای LED قرمز می‌سازد.
- هر دور شامل یک روشن و یک خاموش با تأخیر ۵۰۰ میلی‌ثانیه است.
- پس از پایان حلقه، LED زرد یک ثانیه روشن مانده و خاموش می‌شود.

توضیح عملکرد

در این پروژه، تعداد دفعات چشمک با شمار حلقه کنترل می‌شود. بدنهٔ حلقه پنج بار اجرا شده و LED قرمز را با ریتم ۵۰۰ میلی‌ثانیه‌ای چشمک می‌کند. پس از تکمیل حلقه، نوبت به LED زرد می‌رسد که یک‌بار به مدت یک ثانیه روشن می‌ماند. این نمونه ترکیب «تکرار» و «ترتیب زمانی» را در یک برنامهٔ ساده نشان می‌دهد.



شکل ۵: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۶ چشمک متناوب دو LED با زمان‌های تأخیر متفاوت

صورت مسئله

مداری طراحی کنید که دو LED به پین‌های مختلف آردوینو متصل شوند. این LEDها باید به‌طور متناوب و با زمان‌های تأخیر متفاوت روشن و خاموش شوند.

کد آردوینو

```
int redLED = 9;
int yellowLED = 10;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
}

void loop() {
  digitalWrite(redLED, HIGH);
  delay(1000);

  digitalWrite(redLED, LOW);

  digitalWrite(yellowLED, HIGH);
  delay(2000);

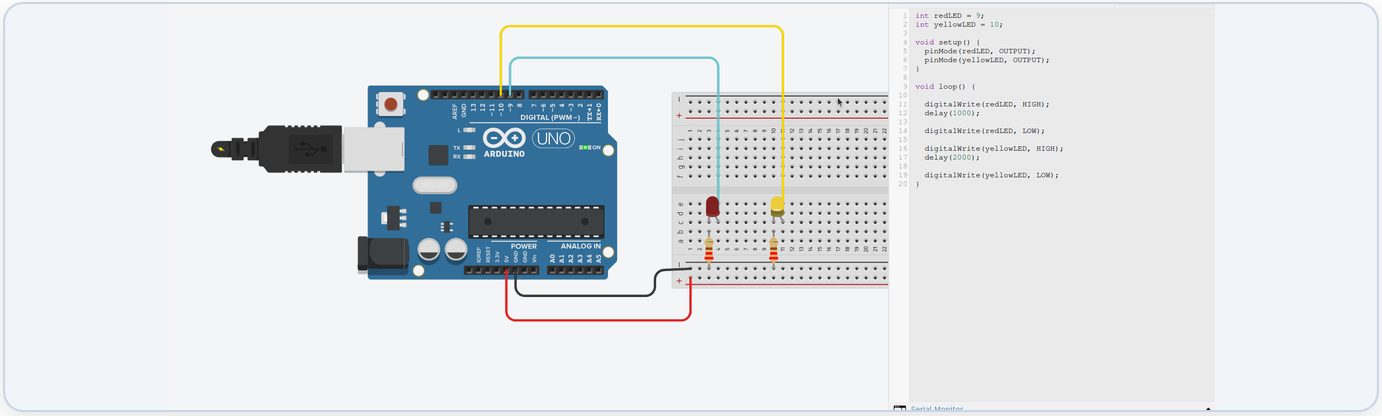
  digitalWrite(yellowLED, LOW);
}
```

توضیح عملکرد

این مدار دو LED را به‌شکل نوبتی و با ریتم‌های ناهمسان روشن می‌کند. نخست LED قرمز به مدت یک ثانیه روشن و سپس خاموش می‌شود؛ بلافاصله LED زرد به مدت دو ثانیه روشن می‌ماند. چون طول دو نیم‌چرخه برابر نیست، الگوی چشمکی نامتقارن و «متناوب با تأخیر متفاوت» ایجاد می‌شود که پایهٔ بسیاری از نشانگرهای زمان‌بندی است.

نکات کلیدی

- LED قرمز یک ثانیه و LED زرد دو ثانیه روشن می‌ماند؛ یعنی دو زمان تأخیر متفاوت.
- در هر لحظه فقط یکی از LEDها روشن است، بنابراین رفتار به‌صورت متناوب دیده می‌شود.
- با تغییر مقادیر delay می‌توان ریتم هر LED را مستقل تنظیم کرد.



شکل ۶: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۷ کنترل تعداد چشمک با متغیر سراسری (Global)

صورت مسئله

با استفاده از یک متغیر سراسری، تعداد دفعات چشمک‌زدن LED را کنترل کنید. مقدار پیش‌فرض را ۵ قرار دهید و سپس آن را تغییر دهید.

کد آردوینو

```
int led = 9;
int blinkCount = 5;

void setup() {
  pinMode(led, OUTPUT);
}

void loop() {
  for (int i = 0; i < blinkCount; i++) {
    digitalWrite(led, HIGH);
    delay(500);

    digitalWrite(led, LOW);
    delay(500);
  }

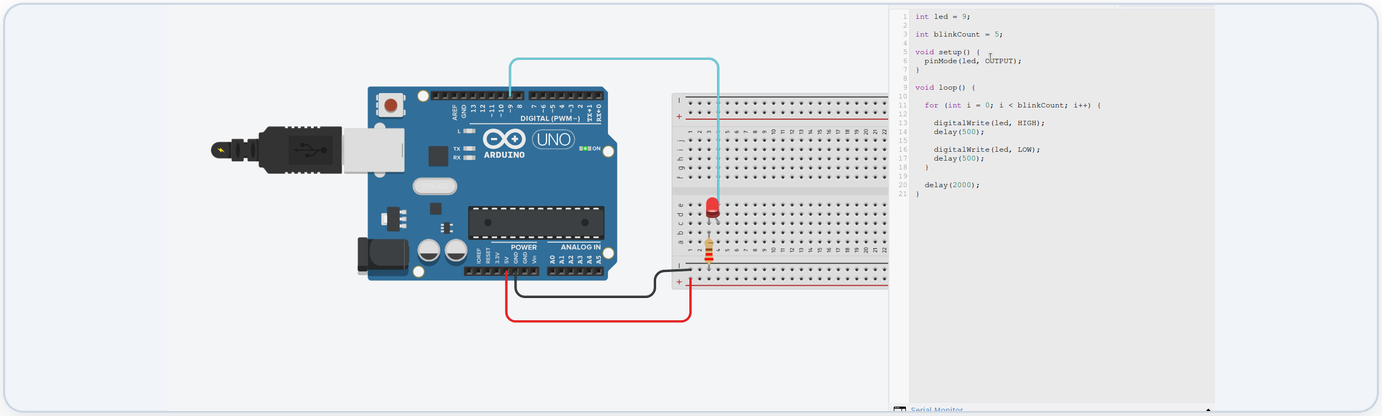
  delay(2000);
}
```

نکات کلیدی

- متغیر `blinkCount` بیرون از توابع تعریف شده و یک متغیر سراسری (Global) است.
- عدد آن در ابتدای برنامه ۵ قرار داده شده و تعداد چشمک‌ها را کنترل می‌کند.
- برای تغییر رفتار مدار کافی است تنها مقدار همین یک متغیر عوض شود.

توضیح عملکرد

در این پروژه، تعداد دفعات چشمک از پیش در کد پخش نشده، بلکه در یک متغیر سراسری به نام blinkCount ذخیره شده است. حلقهٔ for تا رسیدن شمارنده به مقدار این متغیر تکرار می‌شود. با تغییر عدد ۵ به هر مقدار دیگر، بدون دست‌زدن به بقیهٔ کد، تعداد چشمک‌ها عوض می‌شود؛ این همان مزیت استفاده از متغیر در برابر مقادیر ثابت است.



شکل ۷: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۸ روشن شدن LED به مدت ۲ ثانیه و خاموش شدن با دکمه

صورت مسئله

مداری طراحی کنید که یک LED به مدت ۲ ثانیه روشن باشد و پس از آن با فشردن یک دکمه، همان LED خاموش شود.

نکات کلیدی

- روشن‌شدن اولیهٔ LED و تأخیر دو ثانیه در تابع `setup` انجام می‌شود که فقط یک بار اجرا می‌گردد.
- در `loop` وضعیت دکمه پایش می‌شود و با فشردن آن، LED خاموش می‌گردد.
- استفاده از `INPUT_PULLUP` باعث شده حالت فشرده دکمه برابر LOW باشد.

کد آردوینو

```
int led = 9;
int button = 2;

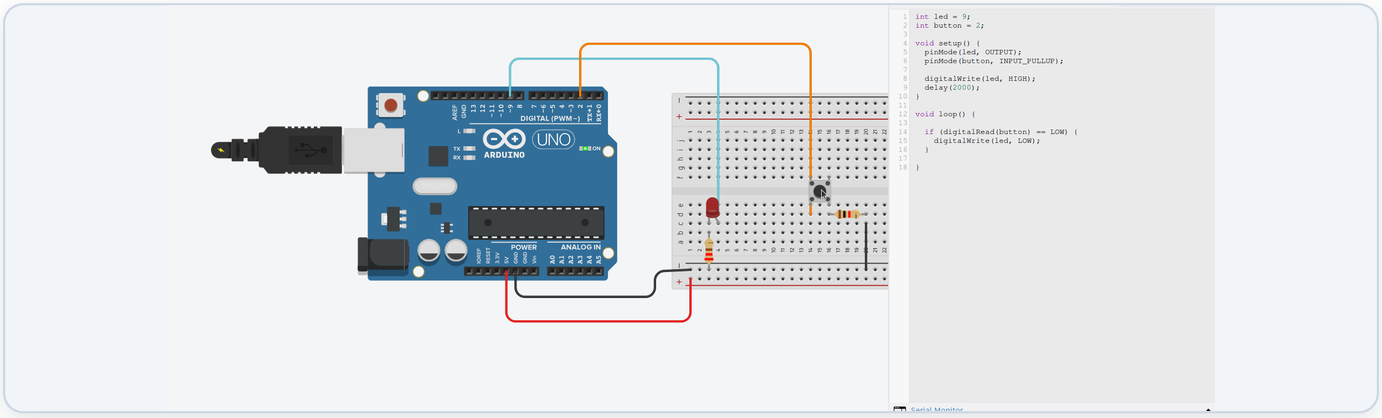
void setup() {
  pinMode(led, OUTPUT);
  pinMode(button, INPUT_PULLUP);

  digitalWrite(led, HIGH);
  delay(2000);
}

void loop() {
  if (digitalRead(button) == LOW) {
    digitalWrite(led, LOW);
  }
}
```

توضیح عملکرد

تفاوت ظریف این پروژه با پروژه‌های پیشین، قرار دادن بخش روشن‌کردن LED در تابع setup است. چون setup تنها یک بار در ابتدای اجرا صدا زده می‌شود، LED بلافاصله روشن شده، دو ثانیه روشن می‌ماند و سپس برنامه وارد loop می‌شود. در loop، آردوینو منتظر می‌ماند تا کاربر دکمه را فشار دهد؛ به‌محض فشردن، فرمان digitalWrite(led, LOW) اجرا شده و LED خاموش می‌شود.



شکل ۸: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۹ تکرار چرخهٔ «۱۰ بار قرمز، سپس یک ثانیه زرد» در حلقه

صورت مسئله

LED قرمز باید ۱۰ بار روشن و خاموش شود، سپس LED زرد به مدت ۱ ثانیه روشن بماند. این عملیات باید در حلقه تکرار شود.

کد آردوینو

```
int redLED = 9;
int yellowLED = 10;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
}

void loop() {
  for (int i = 0; i < 10; i++) {
    digitalWrite(redLED, HIGH);
    delay(500);

    digitalWrite(redLED, LOW);
    delay(500);
  }

  digitalWrite(yellowLED, HIGH);
  delay(1000);

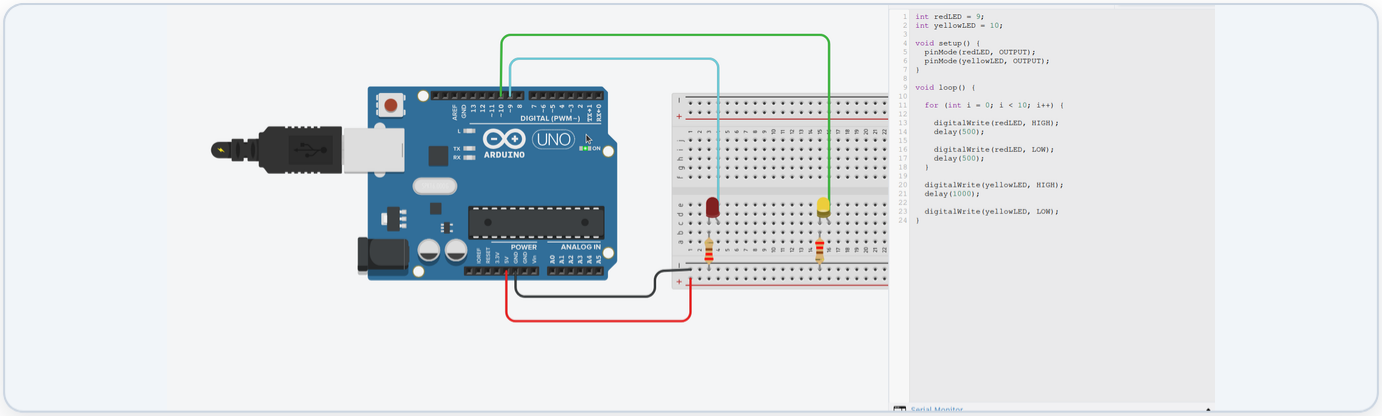
  digitalWrite(yellowLED, LOW);
}
```

نکات کلیدی

- حلقهٔ `for` ده بار LED قرمز را چشمک می‌کند.
- سپس LED زرد یک ثانیه روشن مانده و خاموش می‌شود.
- چون کل این ترتیب در تابع `loop` است، الگو بی‌پایان تکرار می‌شود.

توضیح عملکرد

این پروژه همان الگوی مسئلهٔ ۲ است، اما با تأکید بر تکرار بی‌پایان آن. تابع loop به‌خودی‌خود پس از رسیدن به خط پایانی، از ابتدا اجرا می‌شود؛ بنابراین چرخهٔ «ده بار چشمک قرمز، سپس یک ثانیه روشن بودن زرد» بدون توقف تکرار می‌گردد. این رفتار، تفاوت حلقهٔ for (تکرار محدود) و تابع loop (تکرار نامحدود) را به‌خوبی نشان می‌دهد.



شکل ۹: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۱۰ روشن شدن نوبتی دو LED با چرخهٔ ۵ ثانیه‌ای

صورت مسئله

مداری طراحی کنید که هر ۵ ثانیه یک LED روشن شود و پس از ۵ ثانیه خاموش گردد؛ سپس LED دیگری همین چرخه را تکرار کند.

کد آردوینو

```ino
int led1 = 9;
int led2 = 10;

void setup() {
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
}

void loop() {
  digitalWrite(led1, HIGH);
  delay(5000);

  digitalWrite(led1, LOW);

  digitalWrite(led2, HIGH);
  delay(5000);

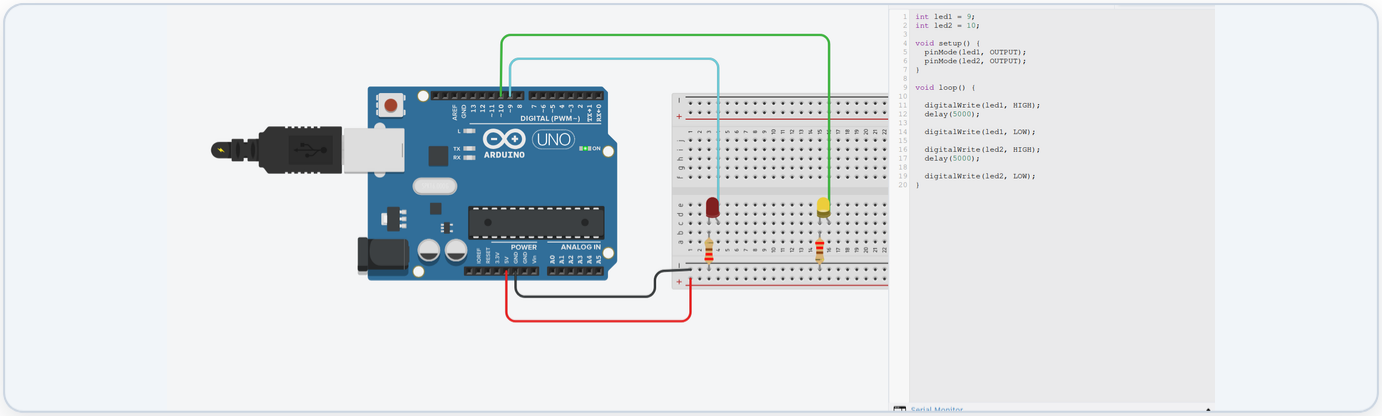
  digitalWrite(led2, LOW);
}
```

نکات کلیدی

- هر LED پنج ثانیه روشن می‌ماند و سپس خاموش می‌شود.
- دستورات به‌ترتیب نوشته شده‌اند تا اول LED یک و بعد LED دو نوبت بگیرد.
- چرخه در تابع `loop` به‌طور مداوم تکرار می‌شود.

توضیح عملکرد

در این مدار دو LED به‌شکل زنجیره‌ای و با زمان برابر روشن می‌شوند. نخست led1 پنج ثانیه روشن مانده و خاموش می‌شود، سپس led2 پنج ثانیه روشن شده و خاموش می‌گردد. چون این توالی در loop قرار دارد، پس از خاموش‌شدن led2 دوباره led1 روشن می‌شود و چرخهٔ پنج‌ثانیه‌ای بی‌پایان ادامه می‌یابد.



شکل ۱۰: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۱۱ روشن شدن LED بر اساس سنسور دما (بالاتر از ۳۰ درجه)

صورت مسئله

با استفاده از یک سنسور دما (مانند LM35/TMP36) دمای محیط را بخوانید؛ اگر دما بیشتر از ۳۰ درجه شد، LED قرمز روشن شود.

نکات کلیدی

- خروجی سنسور به پین آنالوگ A0 وصل است و با `analogRead` عددی بین ۰ تا ۱۰۲۳ خوانده می‌شود.
- عدد خام نخست به ولتاژ و سپس با فرمول `(voltage - 0.5) * 100` به دما تبدیل می‌شود (جبران آفست ۰٫۵ ولت سنسور TMP36).
- اگر دما از آستانهٔ ۳۰ درجه بیشتر شود، LED هشدار روشن می‌گردد.

کد آردوینو

```
int sensorPin = A0;
int ledPin = 9;

float voltage;
float temperature;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int sensorValue = analogRead(sensorPin);

  voltage = sensorValue * (5.0 / 1023.0);

  temperature = (voltage - 0.5) * 100;

  Serial.print("Temperature: ");

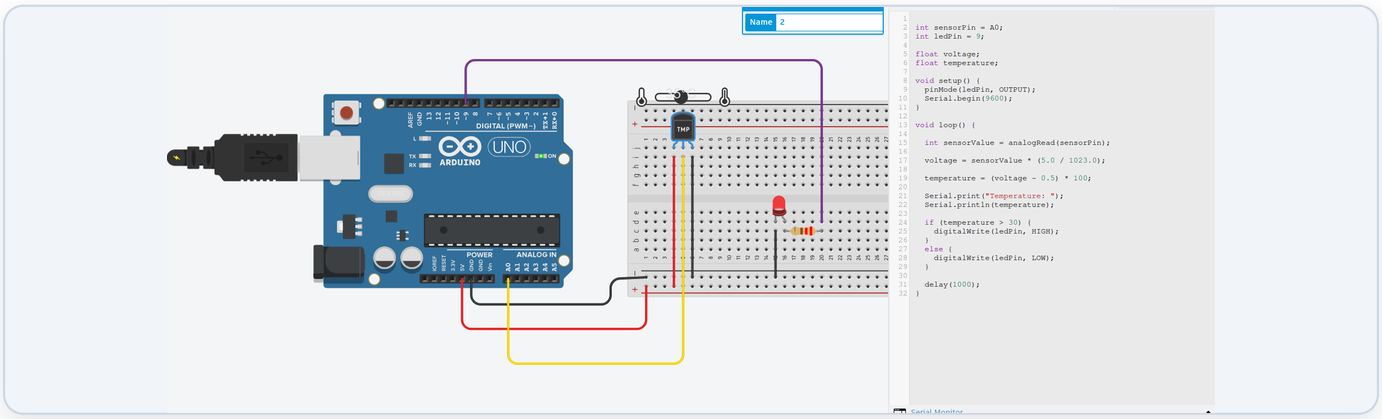
  Serial.println(temperature);

  if (temperature > 30) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }

  delay(1000);
}
```

توضیح عملکرد

این پروژه مفهوم خواندن ورودی آنالوگ و تصمیم‌گیری دیجیتال را با هم ترکیب می‌کند. سنسور دما ولتاژ متناسب با دما تولید می‌کند؛ مبدل آنالوگ‌به‌دیجیتال ۱۰ بیتی آردوینو این ولتاژ را به عددی بین صفر تا ۱۰۳ تبدیل می‌کند. در کد، این عدد به ولتاژ و سپس با کسر آفست ۰٫۵ ولت و ضرب در ۱۰۰ به درجهٔ سلسیوس تبدیل می‌شود. در پایان، مقدار دما روی Serial Monitor چاپ شده و در صورت عبور از ۳۰ درجه، LED قرمز به‌عنوان هشدار روشن می‌شود.



شکل ۱۱: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

مسئلهٔ ۱۲ چشمک دو LED با تأخیر متفاوت با استفاده از millis()

صورت مسئله

مداری طراحی کنید که دو LED با تأخیر متفاوت چشمک بزنند؛ LED اول هر ۱ ثانیه و LED دوم هر ۲ ثانیه روشن و خاموش شود.

کد آردوینو

```ino
int led1 = 9;
int led2 = 10;

unsigned long previousTime1 = 0;
unsigned long previousTime2 = 0;

int state1 = LOW;
int state2 = LOW;

void setup() {
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
}

void loop() {
  unsigned long currentTime = millis();

  if (currentTime - previousTime1 >= 1000) {
    previousTime1 = currentTime;

    state1 = !state1;
    digitalWrite(led1, state1);
  }

  if (currentTime - previousTime2 >= 2000) {
    previousTime2 = currentTime;

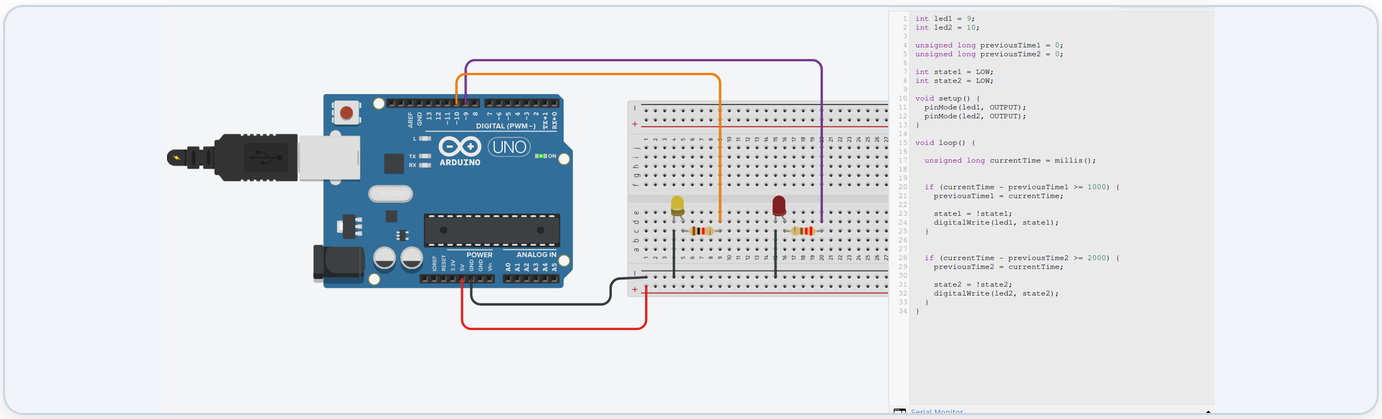
    state2 = !state2;
    digitalWrite(led2, state2);
  }
}
```

نکات کلیدی

- تابع `millis()` زمان سپری‌شده از شروع برنامه (بر حسب میلی‌ثانیه) را برمی‌گرداند و برخلاف delay مانع اجرای بقیهٔ کد نمی‌شود.
- هر LED یک متغیر زمانی و یک وضعیت مستقل دارد تا با ریتم متفاوت (۱ و ۲ ثانیه) کار کند.
- با `state = !state` وضعیت هر LED در هر بازهٔ زمانی معکوس می‌شود.

توضیح عملکرد

این پروژه پیشرفته‌ترین روش زمان‌بندی در آردوینو را نشان می‌دهد. به‌جای استفاده از delay که کل برنامه را متوقف می‌کند، از millis() برای اندازه‌گیری زمان استفاده شده است. در هر دور از loop، زمان جاری خوانده و با زمانِ آخرین تغییر هر LED مقایسه می‌شود؛ اگر اختلاف از آستانهٔ مربوطه (۱۰۰۰ یا ۲۰۰۰ میلی‌ثانیه) بیشتر باشد، وضعیت آن LED معکوس شده و زمان مرجع آن به‌روز می‌شود. چون دو بازهٔ زمانی مستقل‌اند، LEDها با ریتم‌های متفاوت و هم‌زمان چشمک می‌زنند.



شکل ۱۲: محیط شبیه‌سازی Tinkercad Circuits — چیدمان بردبورد، سیم‌کشی و کد اجراشده

۷. نتایج شبیه‌سازی

پس از اجرای هر دوازده تکلیف در محیط Tinkercad، رفتار مشاهده‌شده با انتظار منطقی هر مدار مطابقت داشت. خلاصهٔ نتایج در جدول زیر آمده است:

مسئله	خروجی مورد انتظار	نتیجهٔ مشاهده‌شده	وضعیت
۱	روشن/خاموش مستقل دو LED	هر LED جداگانه با ریتم یک‌ثانیه‌ای	✔ صحیح
۲ و ۹	۱۰ بار قرمز، سپس یک بار زرد	شمارش دقیق با حلقهٔ for	✔ صحیح
۳ و ۸	کنترل LED با دکمه	روشن/خاموش شدن در زمان درست	✔ صحیح
۴ و ۱۰	زمان‌های ۵ و ۳ و ۵ ثانیه	ترتیب و مدت زمان درست	✔ صحیح
۵ و ۶	۵ بار قرمز / تأخیر متفاوت	ریتم‌های مستقل و نامتقارن	✔ صحیح
۷	تعداد چشمک با متغیر سراسری	تغییر فقط با مقدار blinkCount	✔ صحیح
۱۱	هشدار دما بالای ۳۰ درجه	روشن‌شدن LED در آستانه	✔ صحیح
۱۲	دو LED با ۱ و ۲ ثانیه (millis)	زمان‌بندی هم‌زمان و مستقل	✔ صحیح

۸. تحلیل نتایج

در همهٔ مدارها، کلید موفقیت «استقلال خروجی‌ها» بود؛ یعنی تغییر وضعیت یک پین نباید بر پین دیگر اثر بگذارد. حلقهٔ `for` برای شمارش محدود و تابع `loop` برای تکرار نامحدود به کار رفت. در مسئلهٔ ۱۲ نشان داده شد که `millis()` برتری چشمگیری بر `delay()` دارد، زیرا برخلاف delay، برنامه را متوقف نمی‌کند و می‌توان چند LED را هم‌زمان با ریتم‌های متفاوت زمان‌بندی کرد. در مسئلهٔ ۱۱ نیز مقدار آنالوگ سنسور به یک کمیت فیزیکی تبدیل و با یک آستانه مقایسه شد که همان منطق تصمیم‌گیری دیجیتال است.

۹. نتیجه‌گیری

این آزمایش نشان داد که می‌توان چند خروجی دیجیتال را به‌طور کاملاً مستقل از یکدیگر کنترل کرد. با ترکیب فرمان‌های سادهٔ `digitalWrite` و `delay` و به‌کارگیری حلقهٔ `for` و متغیرهای سراسری، رفتارهای متنوعی از جمله چشمک‌زدن با شمارش دقیق، کنترل با دکمه، هشدار دما و زمان‌بندی هم‌زمان چند LED ساخته شد. استفاده از `millis()` نیز روش حرفه‌ای زمان‌بندی بدون توقف برنامه را نشان داد.

۱۰. پیشنهادات

- جایگزین کردن همهٔ `delay()`ها با زمان‌بندی مبتنی بر `millis()` برای اجرای هم‌زمان چند کار.
- افزودن منطق ضدپرش (Debounce) برای دکمه جهت حذف فرمان‌های اشتباه ناشی از پرش مکانیکی.
- به‌کارگیری (PWM) `analogWrite` برای کنترل شدت روشنایی LEDها به‌جای فقط روشن و خاموش.
- استفاده از آرایه برای مدیریت هم‌زمان تعداد بیشتری LED با کدی کوتاه‌تر و خواناتر.

۱۱. منابع

1. جزوهٔ درس آزمایشگاه سیستم‌های دیجیتال، بخش آردوینو (جلسهٔ ۳) — م.ح. سلیمانی الموتی.
2. مستندات رسمی Arduino.cc — Reference: digitalWrite, delay, for, millis, analogRead.
3. مستندات Autodesk Tinkercad Circuits.
4. Margolis, J. *Arduino Cookbook*, O'Reilly Media.

✦ پایان ✦